

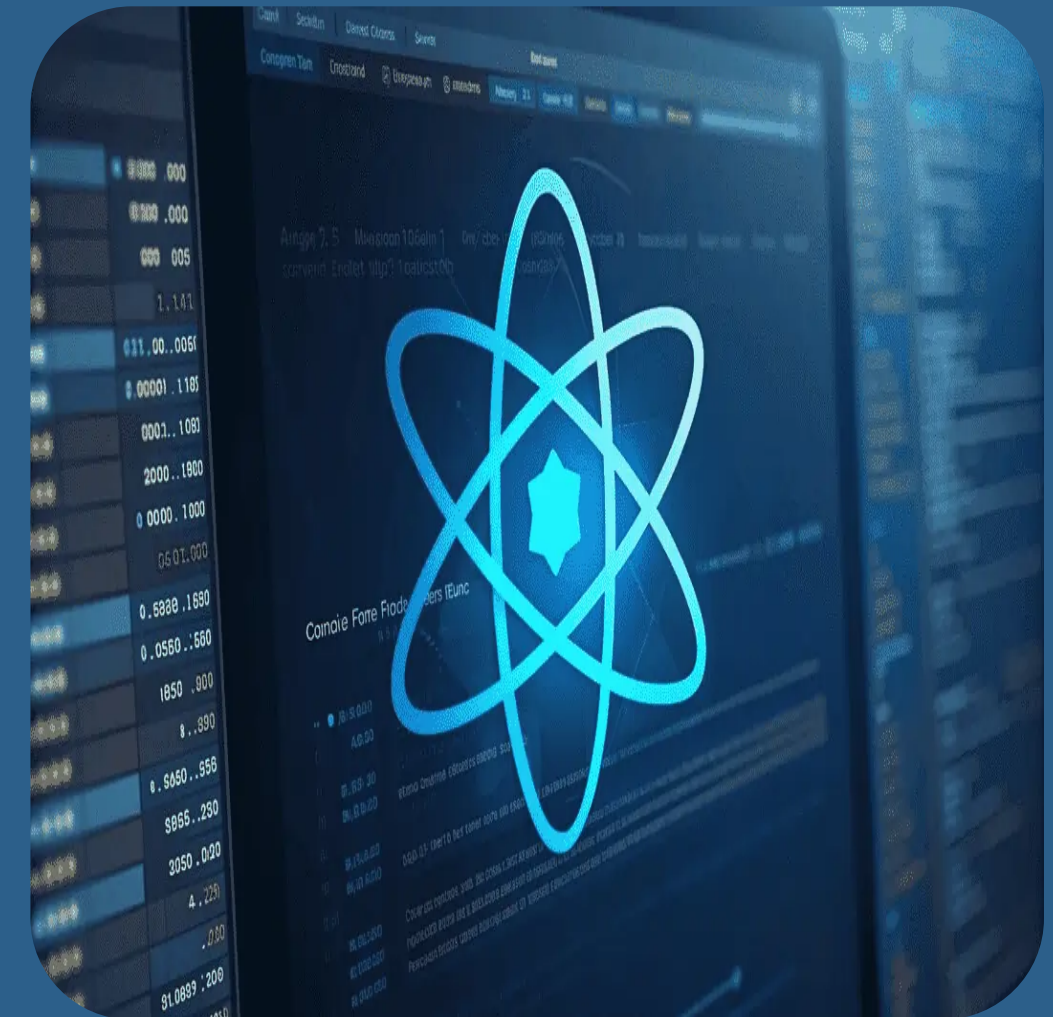
FULL-STACK APPLICATION DEVELOPMENT

Web Application Structure & Authentication Flow

Week 6 Course Materials

Instructor: Dilovan Matini

Course Provided By: CAPA & GIZ



WEEK 6 — FULL-STACK DEVELOPMENT

Topics This Week

Contents



Day 1: Introduction

SPA Concepts, React Router, Navigation

Day 2: Building Apps

Project Structure, Routes, Navigation Links

Day 3: Authentication

Login Flow, Protected Pages, UI
Improvements

This week covers **multi-page application architecture** using React Router, building navigation systems, and implementing **authentication flows** with protected routes. Students will build a complete Student Portal by week's end.

Key Topics: Multi-Page Applications | React Router | Navigation | Authentication Flow | Protected Pages | UI Improvements with Tailwind CSS

DAY 01

Introduction to Multi-Page Applications

Single Page Applications & React Router

Understanding SPA concepts, routing fundamentals, and navigation with React Router

** This session covers the theoretical foundation of multi-page React applications*

Topics Learned

- **React** — Component-based UI library
- **Components** — Reusable UI building blocks
- **Props** — Data passing between components
- **State** — Managing component data
- **Tailwind CSS** — Utility-first styling

This Week's Question

How do large applications organize multiple pages?

PREVIEW: What we will build

App

|—— Home

|—— About

|—— Contact

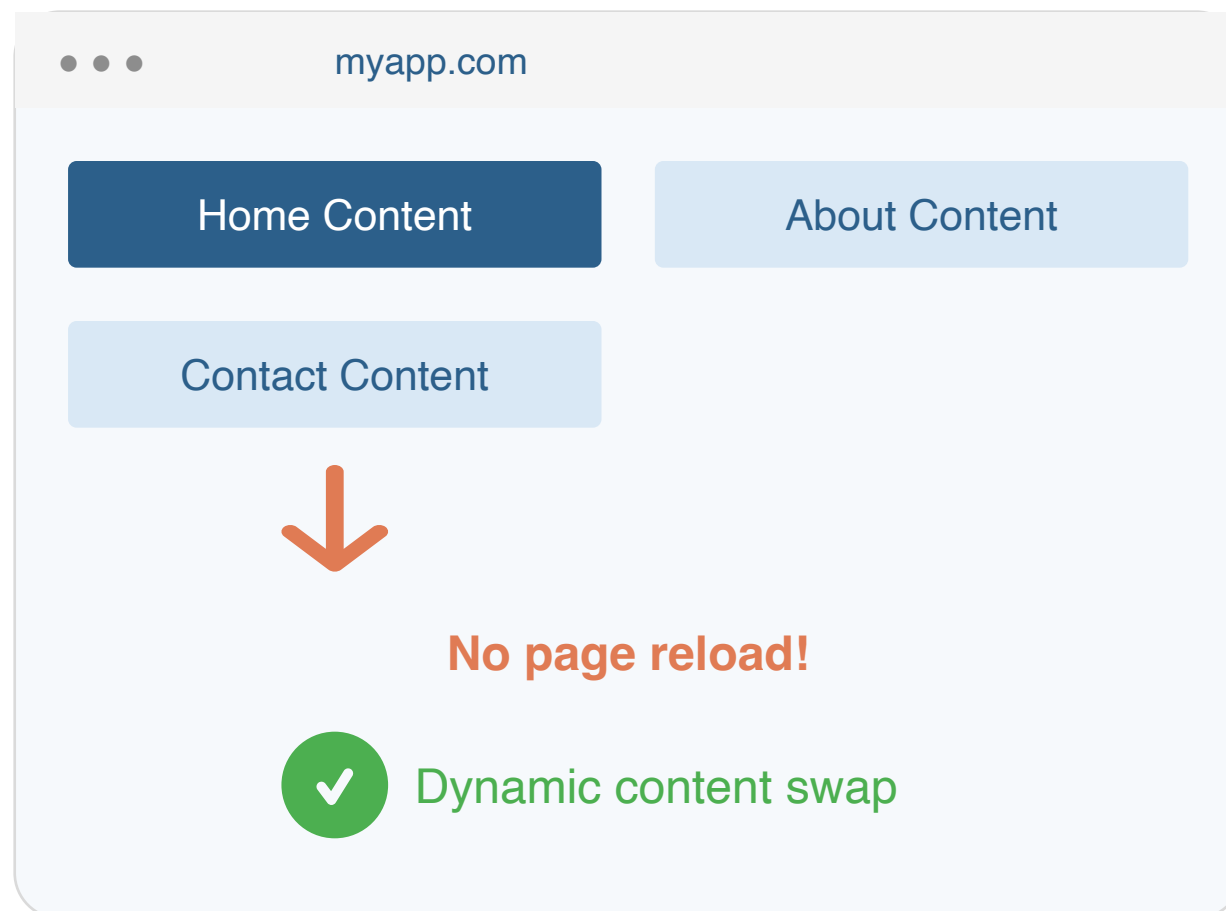
|—— Dashboard (protected)

Routes: /, /about, /contact, /dashboard

React applications are usually Single Page Applications (SPA)

Instead of loading a new HTML page every time, React changes the content **dynamically**.

Single Page Application



Key Characteristics

-  **Fast** — No full page reloads
-  **Smooth** — Seamless user experience
-  **Efficient** — Only data is fetched
-  **Responsive** — Desktop & mobile friendly

The browser loads **one HTML page** and JavaScript handles all content changes dynamically.

Traditional Websites

Multi-Page Application (MPA)

 home.html

 about.html

 contact.html



Each page reloads the entire browser

Server sends a **new HTML file** for every navigation

Slower: Full page reload each time

Heavy: More server requests

React Applications

Single Page Application (SPA)

App (one file: index.html)

├── Home (component)

├── About (component)

├── Contact (component)

└── Dashboard (component)



One application with multiple views

JavaScript handles **content changes**

Faster: No full page reloads

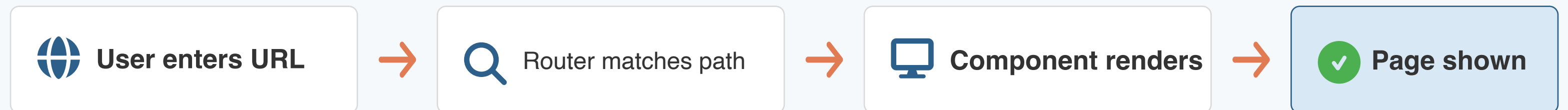
Lighter: Less server load

Routing determines which component should be displayed for a specific URL.

Route Examples



How It Works



Key Idea: The URL in the browser's address bar determines which React component is displayed — all without reloading the page.

React Router is the most popular routing library for React.

Installation

```
$ npm install react-router-dom
```

Run this command in your project directory

Key Features

- **Declarative routing** — Routes as components
- **Nested routes** — Parent/child route structure
- **Dynamic params** — /user/:id patterns
- **Navigation** — Link & useNavigate hooks
- **Protected routes** — Conditional access control

Popular React Router Imports

```
// Core components
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  Navigate
} from 'react-router-dom'
```

Route Mapping

URL Path	Page Component
/	Home Page
/about	About Page
/contact	Contact Page

Navigation Menu Structure

Navbar

Home → /

About → /about

Contact → /contact

Dashboard

Visual: Navigation Bar

Home

About

Contact

Dashboard

Clicking a link changes the page content:

<Home />

Component renders here
without page reload



Hands-On Exercise

Your Task



Create Home page



Create About page



Create Contact page



Create Navigation Menu

Tip: Think about how users will navigate between your pages

DAY 02

Building Multi-Page Applications

Project Structure, Routes & Navigation

Setting up React Router, creating page components, and building navigation links

** This session is hands-on — you will build a working multi-page React application*

Installation

```
$ npm install react-router-dom
```

This adds React Router to your project dependencies

Project Structure

```
src/  
├── pages/  
│   ├── Home.jsx  
│   ├── About.jsx  
│   └── Contact.jsx  
├── App.jsx  
└── main.jsx
```

Why This Structure?

- ✓ **Organized** — Pages in one location
- ✓ **Scalable** — Easy to add new pages
- ✓ **Standard** — Industry best practice
- ✓ **Maintainable** — Clear separation of concerns



Each page is a **separate component file** that exports a default function.

Convention: Page components go in src/pages/



Home.jsx

```
function Home() {  
  return <h1>Home</h1>;  
}  
  
export default Home;
```

About.jsx

```
function About() {  
  return <h1>About</h1>;  
}  
  
export default About;
```

Contact.jsx

```
function Contact() {  
  return <h1>Contact</h1>;  
}  
  
export default Contact;
```

Pattern

Every page component follows the **same simple pattern**:

1. Create a function with the **page name**
2. Return **JSX** for the page content
3. **Export** the function as default

This pattern makes pages reusable and easy to maintain.

Defining Routes in App.jsx

```
import { Routes, Route }  
  from 'react-router-dom';  
  
function App() {  
  return (  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/about" element={<About />} />  
      <Route path="/contact" element={<Contact />} />  
    </Routes>  
  );  
}
```

Wrap with BrowserRouter

```
import { BrowserRouter }  
  from 'react-router-dom';  
  
<BrowserRouter><App /></BrowserRouter>
```

Key Concepts

- i Routes**
Container that holds all Route definitions. Only the first matching Route renders.
- i Route**
Maps a URL path to a React component. When the path matches, the element prop renders.
- i BrowserRouter**
Wraps your app to enable routing. Uses the HTML5 history API for clean URLs.

Remember: BrowserRouter wraps your entire application, typically in main.jsx

Using the Link Component

```
import { Link } from
  'react-router-dom';

<Link to="/">Home</Link>
<Link to="/about">About</Link>
```

Link vs. Anchor Tag

Feature	Link	tag (a)
Page Reload	No reload	Full reload
SPA Routing	Yes	No
Performance	Fast	Slower

Complete Navigation

Home | About | Contact | Dash

How It Works

1. User clicks a Link

↓

2. URL updates (no reload)

↓

3. Router matches new path

↓

4. New component renders



Build a Multi-Page App

Build



Home Page



About Page



Contact Page



Nav Menu

Style



Using **Tailwind CSS**

DAY 03

Authentication & Protected Pages

Authentication Flow & UI Improvements

Login systems, protected routes, public vs private pages, and Tailwind UI polish

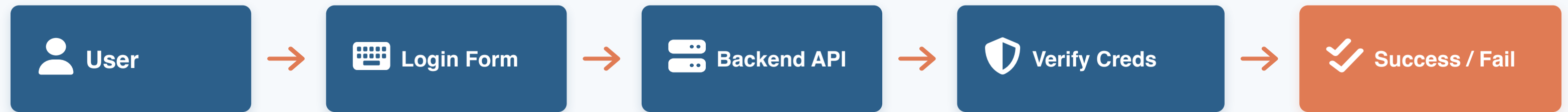
** This session covers security concepts and the mini project — Student Portal*



Authentication answers: Who is the user?

It verifies identity using credentials like **Username** and **Password**.

Authentication Flow



Login Form Fields

 Email address

 Password

Login Button

Key Points

- Authentication **verifies identity**
- Credentials sent to **backend API**
- Server **validates** and responds
- Success = **access granted**
- Failure = **access denied**

All major web applications require **authentication** to access personalized features.



Facebook

Social network requiring login
for news feed, profiles, and
messaging

Authentication Required



Instagram

Photo sharing platform with
login for posts, stories, and DMs

Authentication Required



Gmail

Email service with secure login
for inbox, compose, and
contacts

Authentication Required



Amazon

E-commerce with login for
orders, cart, recommendations,
and Prime

Authentication Required



Public Pages

Accessible to everyone



Home — Landing page



About — Information page



Contact — Contact form

No login needed. Anyone can view these pages.



Protected Pages

Require login to access



Dashboard — User overview



Profile — Personal data



Settings — Preferences

Login required. Unauthenticated users are redirected.

Login Page Fields

-  **Email** — User identifier
-  **Password** — Secret credential
-  **Login Button** — Submit credentials
-  **Token** — Session key returned

Protected Route Logic

Is User Logged In?

YES

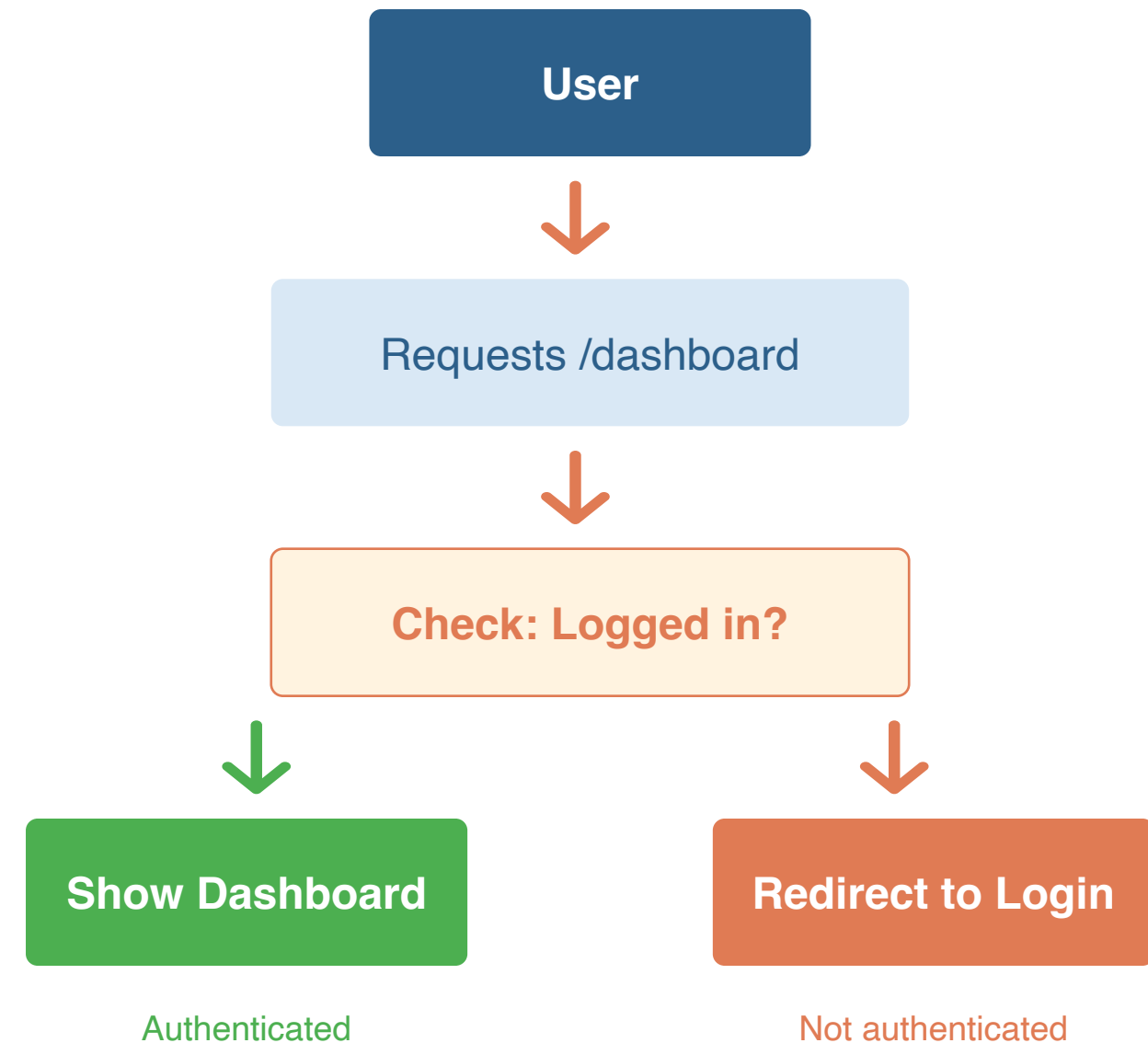
→ Dashboard

OR

NO

→ Login Page

Flow Diagram



Use Tailwind utility classes to create polished, professional interfaces.



Colors

bg-blue-600

text-white



Shape & Shadow

rounded-lg

shadow-lg



Spacing

p-4 (padding)

m-2 (margin)

Focus On



Consistency



Spacing



Readability

Example: `className="bg-blue-600 text-white rounded-lg shadow-lg p-4"` creates a polished button card



Build a Student Portal

Required Pages



Home



About



Login



Dashboard

Features to Implement

Navigation Menu

Tailwind Styling

Protected Dashboard

Login Flow

WEEK 6 WRAP-UP

Topics Learned

React Router

Multi-Page Applications

Navigation

Students can now:

Create multiple pages

Navigate between pages

Authentication Concepts

Protected Pages

UI Improvements

Understand login flow

Organize larger React apps



Next Week: Mobile Development

React Native & Expo